

Ex. No.: 8

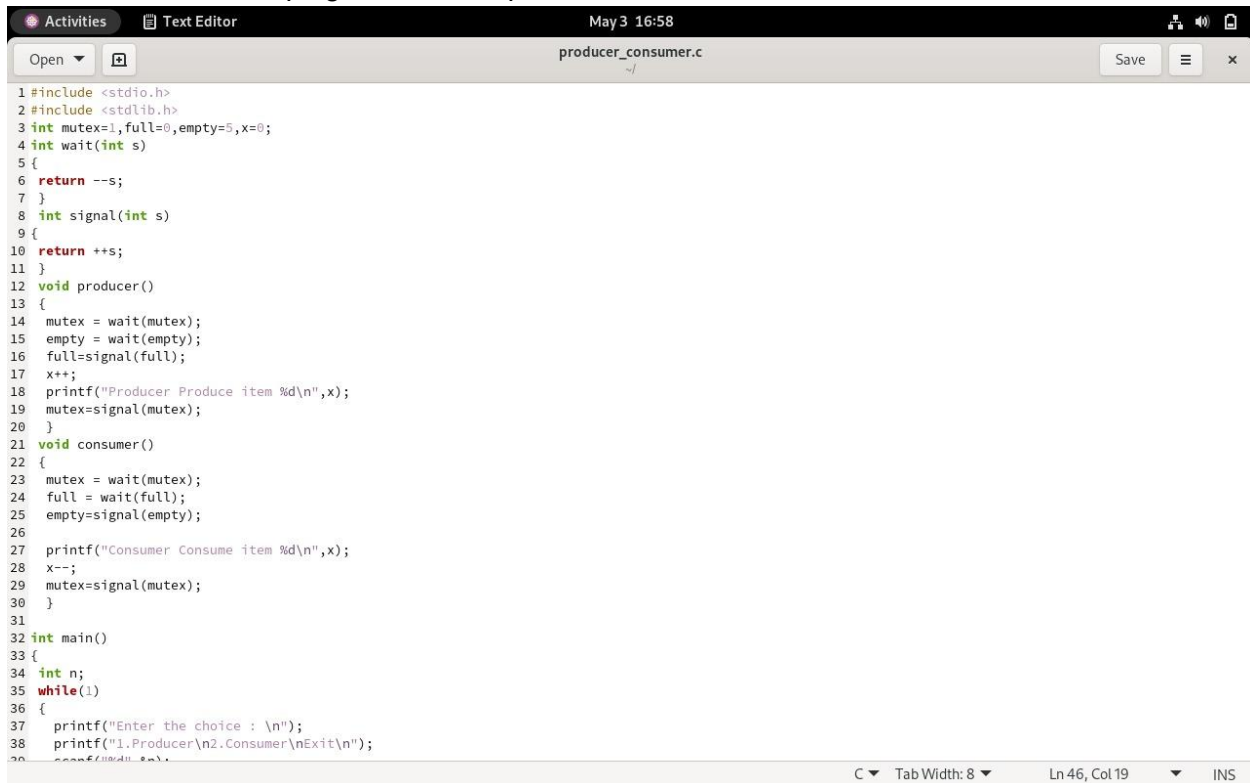
Date:

PRODUCER CONSUMER USING SEMAPHORES

Aim: To write a program to implement solution to producer consumer problem using semaphores.

Algorithm:

1. Initialize semaphore empty, full and mutex.
2. Create two threads- producer thread and consumer thread.
3. Wait for target thread termination.
4. Call sem_wait on empty semaphore followed by mutex semaphore before entry into critical section.
5. Produce/Consume the item in critical section.
6. Call sem_post on mutex semaphore followed by full semaphore before exiting critical section.
7. before exiting critical section.
8. Allow the other thread to enter its critical section.
9. Terminate after looping ten times in producer and consumer Threads each.



```
1 #include <stdio.h>
2 #include <stdlib.h>
3 int mutex=1, full=0, empty=5, x=0;
4 int wait(int s)
5 {
6     return --s;
7 }
8 int signal(int s)
9 {
10    return ++s;
11 }
12 void producer()
13 {
14     mutex = wait(mutex);
15     empty = wait(empty);
16     full=signal(full);
17     x++;
18     printf("Producer Produce item %d\n",x);
19     mutex=signal(mutex);
20 }
21 void consumer()
22 {
23     mutex = wait(mutex);
24     full = wait(full);
25     empty=signal(empty);
26
27     printf("Consumer Consume item %d\n",x);
28     x--;
29     mutex=signal(mutex);
30 }
31
32 int main()
33 {
34     int n;
35     while(1)
36     {
37         printf("Enter the choice : \n");
38         printf("1.Producer\n2.Consumer\nExit\n");
39         printf("%d\n", n);
```

```
Activities Text Editor May 3 16:59
producer_consumer.c
Save x

33
34 int n;
35 while(1)
36 {
37     printf("Enter the choice : \n");
38     printf("1.Producer\n2.Consumer\nExit\n");
39     scanf("%d",&n);
40     switch(n)
41     {
42     case 1 :
43         if(mutex==1 && empty!=0)
44         {
45             producer();
46         }
47         else
48         {
49             printf("Buffer is Full !...\n Producer cannot produce \n");
50         }
51         break;
52     case 2 :
53         if(mutex==1 && full!=0)
54         {
55             consumer();
56         }
57         else
58         {
59             printf("Buffer is Empty!...\n Consumer cannot consume \n");
60         }
61         break;
62     case 3 :
63         printf("Exiting From the CPU...\n");
64         return 0;
65         break;
66     default:
67         printf("Invalid Choice \n Enter Another Choice\n");
68         break;
69     }
70 }
71
```

```
[PraveenKumar212@localhost ~]$ gcc producer_consumer.c -o producer_consumer
[PraveenKumar212@localhost ~]$ ./producer_consumer
```

```
Enter the choice :
1.Producer
2.Consumer
Exit
1
Producer produce item 1
Enter the choice :
1.Producer
2.Consumer
Exit
1
Producer produce item 2
Enter the choice :
1.Producer
2.Consumer
Exit
1
Producer produce item 3
Enter the choice :
1.Producer
2.Consumer
Exit
1
Producer produce item 4
Enter the choice :
1.Producer
2.Consumer
Exit
1
Producer produce item 5
Enter the choice :
1.Producer
2.Consumer
Exit
1
Buffer is Full !....
Producer cannot produce
Enter the choice :
1.Producer
2.Consumer
Exit
```

```
[PraveenKumar212@localhost ~]$ gcc producer_consumer.c -o producer_consumer
[PraveenKumar212@localhost ~]$ ./producer_consumer
Enter the choice :
1.Producer
2.Consumer
Exit
1
Producer produce item 1
Enter the choice :
1.Producer
2.Consumer
Exit
1
Producer produce item 2
Enter the choice :
1.Producer
2.Consumer
Exit
1
Producer produce item 3
Enter the choice :
1.Producer
2.Consumer
Exit
1
Producer produce item 4
Enter the choice :
1.Producer
2.Consumer
Exit
1
Producer produce item 5
Enter the choice :
1.Producer
2.Consumer
Exit
1
Buffer is Full !....
Producer cannot produce
Enter the choice :
1.Producer
2.Consumer
Exit
█
```